



One-class graph neural networks for anomaly detection in attributed networks

Xuhong Wang¹ · Baihong Jin² · Ying Du¹ · Ping Cui¹ · Yingshui Tan² · Yupu Yang¹

Received: 7 July 2020 / Accepted: 11 March 2021 / Published online: 25 March 2021
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2021

Abstract

Nowadays, graph-structured data are increasingly used to model complex systems. Meanwhile, detecting anomalies from graph has become a vital research problem of pressing societal concerns. Anomaly detection is an unsupervised learning task of identifying rare data that differ from the majority. As one of the dominant anomaly detection algorithms, one-class support vector machine has been widely used to detect outliers. However, those traditional anomaly detection methods lost their effectiveness in graph data. Since traditional anomaly detection methods are stable, robust and easy to use, it is vitally important to generalize them to graph data. In this work, we propose one-class graph neural network (OCGNN), a one-class classification framework for graph anomaly detection. OCGNN is designed to combine the powerful representation ability of graph neural networks along with the classical one-class objective. Compared with other baselines, OCGNN achieves significant improvements in extensive experiments.

Keywords Graph anomaly detection · Graph neural network · Anomaly detection · Unsupervised learning

1 Introduction

Nowadays, graph-structured data are increasingly used to model complex systems, ranging from social media networks [22], traffic networks [41] to financial nets [36]. A

graph is a structure amounting to a set of nodes V in which some pairs of them are related by edges E . Through the introduction of edges between data instances, graph provides a powerful tool for effectively capture long-range correlations between them. Meanwhile, detecting anomalies from graph has become a vital research problem of pressing societal concerns [1]. Firstly, graph data with anomalies can disrupt the performance of machine learning algorithms and bring serious consequences. Secondly, graph anomaly detection (GAD) has significant applications in many security-related domains, e.g., discovering suspicious financial transactions, monitoring traffic jams and unveiling malicious users in social networks.

Anomaly detection (or outlier detection) can be regarded as the task of identifying rare data items that differ from the majority of the data [27]. As one of the dominant anomaly detection algorithms, hypersphere learning approaches, e.g., Support Vector Data Description (SVDD) [32], typically try to define a boundary around the given normal class data. Afterward, whether the unknown data point is anomalous is determined by their location with respect to the boundary.

Although those anomaly detection methods have been applied in diverse domains [27], they may lose their

✉ Xuhong Wang
wang_xuhong@sjtu.edu.cn

Baihong Jin
bjin@berkeley.edu

Ying Du
duying@sjtu.edu.cn

Ping Cui
cuiping@sjtu.edu.cn

Yingshui Tan
tys@berkeley.edu

Yupu Yang
ypyang@sjtu.edu.cn

¹ The Ministry of Education Key Laboratory of System Control and Information Processing, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China

² Department of Electrical Engineering and Computer Sciences, University of California, Berkeley, USA

effectiveness in graph data. Those methods can only handle Euclidean data (e.g., images, texts and tabular data) rather than non-Euclidean data (Graph). In the case of Euclidean data, data are treated as independent points lying in a multi-dimensional Euclidean space; however, nodes in graph may exhibit inter-dependencies, which means they are inherently related to one another. Traditional anomaly detection methods are widely used and easy to use, and their application to graph data is almost an inevitable evolution. Graph embedding methods, such as DeepWalk [26], Struc2Vec [28] and Tree2Vector [42], can extract node relationship information as a fixed-length feature vector. Afterward, traditional anomaly detection methods can be trained from that vector. However, these two-stage methods are decoupled in the sense that the graph embedding learning is task agnostic and not customized for GAD, which has a limited and indirect contribution to detecting anomalous nodes.

Previous GAD works [1] have achieved empirical success, which usually adopt graph matrix factorization to locate the anomaly nodes or find the target community at first and then detect outliers in a predefined subspace. However, several challenges exist in GAD tasks: (i) data nonlinearity. The node feature and the interactions among nodes are often highly nonlinear while many GAD methods rely on linear mechanisms. (ii) Computational complexity. In the era of data deluge, real graphs can easily have millions of nodes and edges. The computational overhead of classical GAD methods discourages their applicability to large-scale graphs. (iii) Researchers and developers are proficient in simple off-the-shelf Euclidean anomaly detection techniques. Previous statistical graph feature-based GAD methods increase their understanding and deployment costs, which greatly limits their development in the graph mining community.

Graph neural networks (GNNs) [40] are a significant stride to operate directly on graph-structured data, and a promising method to address these above problems. GNNs are essentially a message passing (or neighborhood node aggregation) scheme, where each node aggregates feature information of its neighbors to compute its new feature vector. After multiple iterations of information aggregation, the feature vector of a node will capture the structural information among the node's neighborhood. GNNs have nonlinear activation functions and parallelization capabilities, which can address the data nonlinearity and the computational complexity issues, respectively. Even though GNNs have demonstrated outstanding performance in many graph mining tasks [22, 36, 41], it remains unclear how to exploit their potentiality for GAD problem.

To the best of our knowledge, little progress has been made to apply GNNs to GAD tasks. Ding et al. [7] and Li et al. [16] tried to solve the problems using graph

autoencoder (GAE) [13]. They used graph convolutional network (GCN) [14], a simple form of GNNs, to construct the encoder and decoder, so that the node embedding can be extracted from the graph structure information as well as node attributes. However, GAE methods have the objectives of learning low-dimension node embedding and reconstructing the node attributes and connection relationship among nodes, which does not target GAD directly.

There needs to be an end-to-end, easy-to-understand and powerful GAD technology. In this work, we propose one-class graph neural network (OCGNN), a hypersphere learning framework for GAD. As shown in Fig. 1, OCGNN is designed to combine the powerful representation ability of GNNs with the classical hypersphere learning objective to detect anomalies. Because the topology information on graphs is automatically extracted by the GNNs, users of OCGNN can handle GAD problems without deep understandings of graph theories. Furthermore, OCGNN is an end-to-end approach because the node representation learned by OCGNN is highly related to the GAD task, which means the embeddings learned from OCGNN is more friendly to the downstream anomaly detection tasks than those learned from DeepWalk. By aggregating the neighborhood information, an OCGNN computes the embedding of each observable node. Meanwhile, hypersphere learning objective aims at minimizing the volume of a hypersphere that encloses those embedding vectors [29]. An anomalous node is defined by whether the location of its embedding is out of the hypersphere.

Specifically, our contributions are listed as follows.

1. We propose a novel end-to-end OCGNN framework, which aims at applying the powerful representation ability of GNNs into graph anomaly detection tasks. To the best of our knowledge, this is one of the first GNN-based works in this scenario.
2. A hypersphere learning objective is proposed to drive the training of the GNN, thus OCGNN is a natural extension of one-class support vector machine (OCSVM) in the field of graph data. People can use OCGNN to handle GAD problem without any complicated graph theory, which will greatly facilitate the development of graph mining techniques and applications.
3. The source code of our OCGNN has been released at a repository.¹ Besides, we provide up to 14 different baseline models, as well as a function for generating GAD datasets. All these can be run automatically with simple python commands.

The rest of this paper is organized as follows. The related works are introduced in Sect. 2. Section 3 elaborates our

¹ <https://github.com/WangXuhongCN/OCGNN>.

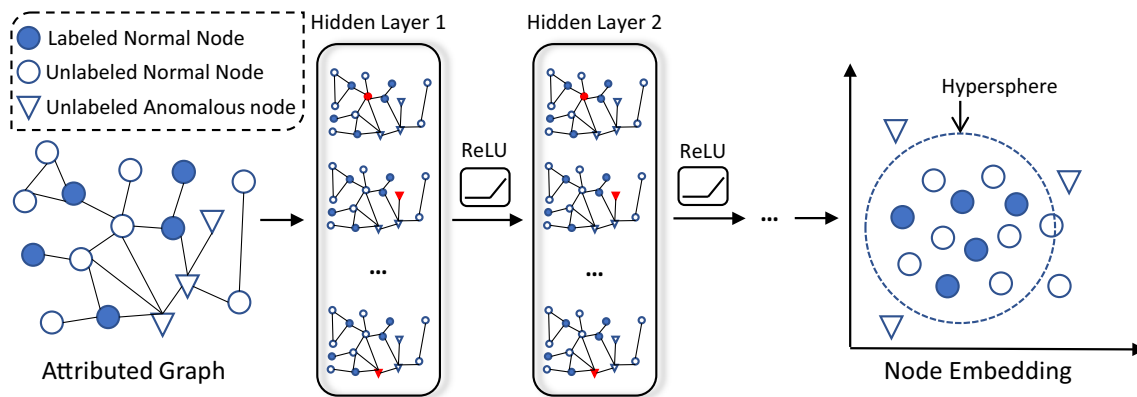


Fig. 1 The overall framework of the proposed OCGNN. Guided by the proposed hypersphere learning objective, OCGNN learns to map the nodes into the embedding space and enclose the embeddings of normal nodes into a hypersphere centered at vector c and with radius

one-class graph neural network method. In Sect. 4, experiments on three real-world datasets are conducted to show the competitiveness and efficiency of our method. Finally, in Sect. 5, we propose our conclusions.

2 Related work

In this section, we briefly review related work in three aspects: (1) traditional anomaly detection, (2) graph neural networks and (3) graph-based anomaly detection.

2.1 Anomaly detection in Euclidean data

Era of data deluge has grown the interest for anomaly detection techniques, especially for spotting anomalies in collections of multi-dimensional data points [27]. Owing to the prohibitive cost for accessing ground truth anomalies, most of the existing methods are unsupervised, which can be divided into four main categories.

(1) Distance-based approaches, such as local outlier factor (LOF) [3] and Isolation Forest (IForest) [17], assume that normal data are tightly clustered, while anomaly data are located far from their nearest neighbors. (2) Boundary-based approach, e.g., SVDD [32], typically tries to define a boundary around the normal class data. An unknown data that located outside the boundary is defined as an outlier. (3) Reconstruction-based approach assumes that anomalies cannot be effectively reconstructed from low-dimensional projections. In this category, PCA [21] and its modified versions models [4, 34] are widely used and effective techniques to detect anomalies. (4) Deep learning methods, such as autoencoder (AE) [5, 43], Variational Autoencoders (VAE) [12, 38] and Generative Adversarial Networks (GAN) [10, 18], learn a high-

dimensional probability distribution in an anomaly-free dataset and then detect outliers deviated from that distribution. Since it becomes nearly impossible for the shallow anomaly detection to find outliers efficiently as the volume of data increases, there is a growing interest in deep learning-based anomaly detection algorithms. Besides, Ruff et al. [29] extended the shallow SVDD method to deep version, which shows promising results on image data.

However, those four types of method mentioned above can only handle graph data with the precondition of graph embedding algorithms. In this two-stage case, the prepossess node embedding is not learned with an anomaly detection target, which results in bad performance. OCGNN integrates two stages to provide an end-to-end learning method.

2.2 Graph neural networks

Graph neural network (GNN) is a promising method to operate directly on non-Euclidean data. In GNN's neighborhood aggregation scheme, each node aggregates feature information of its neighbors to compute its new feature vector layer by layer. After multiple iterations of information aggregation, the feature vector of a node will capture the structural information among the node's neighborhood. The embedding representation learned by GNNs can be used for downstream tasks such as node classification, link prediction, graph classification and so on.

According to the ways that information is aggregated, GNNs can be divided into two forms: isotropic model and anisotropic model. Isotropic model assumes that each neighbor contributes equally to the update of the central node. The typical isotropic GNNs are Graph Convolutional

network (GCN) [14], GraphSAGE [11] and graph isomorphism network (GIN) [40]. On the other hand, anisotropic models assume that the importance of each neighbor to the central node is not equal, and use different weights to aggregate information from neighboring nodes through some mechanism (e.g., attention mechanism in graph attention network (GAT) [33] or gating mechanism in residual gated graph ConvNets (GatedGCN) [2].

Besides, graph Autoencoder (GAE) [13] is a natural extension of traditional AE to graph, which aim to learn low dimensional node vectors via a GCN built encoder and reconstruct the graph structure (adjacency matrix $\mathbf{A}$) via a link prediction decoder.

Nevertheless, all these GNN methods focus on learning universal node representation that can generalize to major downstream tasks, such as node classification or link prediction. It remains unclear how to exploit the potential of GNNs for GAD task. OCGNN model is specifically trained for graph anomaly detection task.

2.3 Anomaly detection on attributed graph

Recently, we have witnessed an increasing amount of efforts in spotting anomalies on attributed networks, and the existing methods can be divided into three categories: structure based, subspace based and residual based.

(1) Early GAD methods assume that anomaly nodes are always involved in dense communities. They use graph structure information only such as community (common neighbors) analysis [8] and ego-net features [24] to detect anomalies on the graph. (2) Besides that, researchers found that anomalies in attributed networks could be spotted in only a subset of node attributes [25, 30]. Although these subspace-based methods can achieve superior results, they use little network structure information, which undoubtedly reduces the learning efficiency of graph data. (3) Recently, residual analysis-based methods [15, 23] and GAE methods [7, 16] leverage the graph structure information as well as its coherence with nodal attributes and have shown an increasing influence. Generally, the former methods use matrix decomposition characterizing and residual estimating techniques, but those shallow residual analysis mechanisms are not enough for today's computing demand for large-scale and nonlinear data. On the other hand, GAE methods have the objectives of learning low-dimension node embedding and reconstructing the node attributes and connection relationship among nodes, which has an indirect contribution to anomaly detection on attributed graphs.

Thanks to the ability of GNN to automatically extract graph structural features, users can handle GAD tasks using OCGNN without deep understandings of graph theories. At the same time, the traditional graph anomaly detection

introduces quite a lot of graph-related hyperparameters, while OCGNN has very few hyperparameters and is robust.

3 One-class graph neural network

As shown in Table 1, for easy retrieval, we summarize the commonly used notations of this paper.

3.1 Definitions

In this section, we introduce notations and definitions of node-wise one-class classification and hypersphere learning.

Definition 1 *Attributed graph*: an attributed graph is $\mathcal{G} = (\mathbf{V}, \mathbf{E}, \mathbf{X})$, where $\mathbf{V} = \{v_1, \dots, v_N\}$ is the set of $N = |\mathbf{V}|$ nodes, $\mathbf{E} \subseteq \mathbf{V} \times \mathbf{V}$ is the set of $M = |\mathbf{E}|$ edges between nodes and $\mathbf{X} \in \mathcal{R}^{N \times D}$ denotes the features of the N nodes. The graph structure can be represented by an adjacency matrix $\mathbf{A} \in \mathcal{R}^{N \times N}$, where $\mathbf{A}_{i,j} = 1$ if there is an edge between node v_i and v_j , otherwise $\mathbf{A}_{i,j} = 0$.

Definition 2 *Anomaly detection*: given the anomaly-free training dataset $\{x_i, i = 1, \dots, K\}$, the model is trained to describe the boundary of normal data, and then model produces the anomaly score $S(x_u)$ for an unseen data point x_u . A data point with high anomaly score is defined as an outlier.

Definition 3 *Node-wise graph anomaly detection*: given a graph, all nodes $\mathbf{X}$, edges $\mathbf{A}$ and only part of the node labels are available for training. Afterward, the labels of the remaining unlabeled nodes are predicted by the trained model. All nodes for training belong to one class (normal),

Table 1 Commonly used notations

Notations	Descriptions
$\mathbf{V} = \{v_1, \dots, v_N\}$	The set of N nodes in a graph
$\mathbf{V}_{tr} \subseteq \mathbf{V}, \mathbf{V}_{tr} = K$	The set of K training nodes
$\mathbf{X} \in \mathcal{R}^{N \times D}$	Node feature matrix
$\mathbf{A} \in \mathcal{R}^{N \times N}$	Adjacency matrix
$g(\mathbf{X}, \mathbf{A}; \mathcal{W})$	A graph neural network
$\mathcal{W} = \{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}\}$	Trainable weights set of g
$\mathbf{W}^{(l)}$	Network weights of l -th GNN layer
$\mathbf{Z} \in \mathcal{R}^{N \times F}$	Node embedding matrix
$[\cdot]^+ = \max(0, \cdot)$	Nonnegative operator
$c \in \mathcal{R}^F$	The hypersphere center
$r \in \mathcal{R}^+$	The hypersphere radius

while the remaining testing nodes belong to two classes (normal and abnormal).

3.2 Hypersphere learning

Hypersphere learning is originally proposed in SVDD [32], which aims at learning a compact hypersphere boundary to cover all training data and detecting which (new) objects resemble this training set. In the application of anomaly detection, the training data are all normal, therefore hypersphere learning model can obtain a description boundary of normal data, then outliers can be precluded.

Let $\mathcal{X} \subseteq \mathcal{R}^d$ be the data space, $\phi_k: \mathcal{X} \rightarrow \mathcal{F}_k$ be a mapping function from data space $\mathcal{X}$ to reproducing kernel Hilbert space $\mathcal{F}_k$, where $k: \mathcal{X} \times \mathcal{X} \rightarrow [0, \infty)$ is a positive-definite kernel function. The objective of SVDD is to describe the smallest hypersphere, in feature space $\mathcal{F}_k$, with center $c \in \mathcal{F}_k$ and radius $r > 0$ that encloses the majority of the training data. Given a training dataset $\mathcal{X}_K = \{x_i \in \mathcal{X}, i = 1, \dots, K\}$, SVDD solves the primal problem:

$$\min_{r, c} r^2 + \frac{1}{\beta K} \sum_{i=1}^K \xi_i \quad (1)$$

$$\text{s.t. } \|\phi_k(x_i) - c\|_{\mathcal{F}_k}^2 \leq r^2 + \xi_i, \quad \xi_i \geq 0, \quad \forall i$$

where ξ_i are nonnegative slack variables to allow the contamination of outliers in the training dataset. The data points $\phi_k(x_i) \in \mathcal{F}_k$ are not strictly inside the hypersphere, but the data located too away from the boundary should be penalized. The hyperparameter $\beta \in (0, 1]$ controls the trade-off between the sphere volume and the penalties. After minimizing Eq. (1), the center c and the radius r can be obtained, while ξ_i is not a learnable parameter. Data point x_i that falls outside the hypersphere, i.e., $\|\phi_k(x_i) - c\|_{\mathcal{F}_k}^2 > r^2$, is defined as an outlier.

3.3 Learning objective

With classical hypersphere learning objective, an SVDD model builds a minimum volume hypersphere estimation of given data. However, our OCGNN learns useful node representations together with the hypersphere learning objective. To achieve this goal, we introduce GNN that is jointly trained to learning node embeddings by considering both node attributes and relationships and to keep the embeddings into a minimum hypersphere.

GNNs consider both the node attributes $\mathbf{X} \in \mathcal{R}^{N \times D}$ and the adjacency matrix $\mathbf{A} \in \mathcal{R}^{N \times N}$ (some GNN models use neighbor aggregation to replace the role of $\mathbf{A}$) when learning the node embedding vectors $\mathbf{Z} \in \mathcal{R}^{N \times F}$. Hence,

We use $g(\mathbf{X}, \mathbf{A}; \mathcal{W})$ to represent a GNN with layer-wise trainable weights set $\mathcal{W} = \{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}\}$ where $L \in \mathcal{N}$ is the number of hidden layers. For a specific l th layer, the forward propagation rule of GNN can be summarized as:

$$\mathbf{H}^{(l+1)} = g(\mathbf{H}^{(l)}, \mathbf{A}; \mathbf{W}^{(l)}), \quad (2)$$

where $\mathbf{H}^{(l)}$ is the input for the l th GNN layer and $\mathbf{H}^{(l+1)}$ is the output after this layer. Note that the node attributes $\mathbf{X}$ is the input of the first layer, which equals to $\mathbf{H}^{(0)}$. The embedding matrix of the nodes $\mathbf{Z}$ is the final output $\mathbf{H}^{(L)}$. Thanks to the powerful GNN node embedding, it helps the performance improvement of down-stream tasks, such as node-level and graph-level classification and link prediction.

The aim of OCGNN is to jointly learn the net parameters $\mathcal{W}$ and minimize the volume of the data description hypersphere which is characterized by a radius $r \in \mathcal{R}^+$ and a center $c \in \mathcal{R}^F$. Given a graph defined by $(\mathbf{X}, \mathbf{A})$ and the set of K training nodes $\mathbf{V}_{tr} \subseteq \mathbf{V}$, where $K = |\{i: v_i \in \mathbf{V}_{tr}\}|$, the objective of OCGNN is set as:

$$\mathcal{L}(r, \mathcal{W}) = \frac{1}{\beta K} \sum_{v_i \in \mathbf{V}_{tr}} [\|g(\mathbf{X}, \mathbf{A}; \mathcal{W})_{v_i} - c\|^2 - r^2]^+ + r^2 + \frac{\lambda}{2} \sum_{l=1}^L \|\mathbf{W}^{(l)}\|^2. \quad (3)$$

In the forward propagation, OCGNN receives all nodes and edges information in a graph and then output a node embedding matrix $\mathbf{Z} = g(\mathbf{X}, \mathbf{A}; \mathcal{W})$, $\mathbf{Z} \in \mathcal{R}^{N \times F}$. However, only K node embeddings $\{\mathbf{Z}_{v_i}, v_i \in \mathbf{V}_{tr}\}$ are used in the loss function computation.

The first term of Eq. (3) is a penalty for node embeddings lying out of the hypersphere, i.e., if the distance between an embedding vector and the center c is greater than the radius r . The hyperparameter $\beta \in (0, 1]$ controls the trade-off between the sphere volume and the penalties (we will discuss it later). As in classical SVDD, the second term, minimizing r^2 , is to minimize the volume of sphere. The last term is the weight decay regularizer on the OCGNN network parameters $\mathcal{W}$ with a hyperparameter $\lambda > 0$.

The objective of OCGNN is to generate the node embeddings that are closed the center c of the sphere. Since the training nodes are all normal, OCGNN will extract the common factors of the given nodes. As a result, the description boundary of normal nodes can be obtained and the anomalous nodes can be detected.

For a node v_i in the given graph, its anomaly score $S(v_i)$ can be defined by the location of the embedding respect to the sphere: $S(v_i) = \|g(\mathbf{X}, \mathbf{A}; \mathcal{W}^*)_{v_i} - c\|^2 - r^{*2}$. If $S(v_i) > 0$, the node v_i is anomalous, otherwise it is a normal node.

Note that the network parameters $\mathcal{W}^*$ and the learned radius r^* can characterize a trained OCGNN model. The memory complexity of OCGNN is very low because no data need be stored for model prediction.

To clearly explain the role of β in the training process, we define $\xi = \frac{1}{2} \sum_{l=1}^L \|\mathbf{W}^{(l)}\|^2$ and $d_{v_i} = \|g(\mathbf{X}, \mathbf{A}; \mathcal{W}) - c\|_{v_i}^2$ where $i = 1, \dots, K$. Assuming that there is a node subset $\mathbf{V}_o \subset \mathbf{V}_{tr}$ whose nodes are mapped outside of the sphere by OCGNN, consisting $K_o = |\{i : d_{v_i} > r^2, v_i \in \mathbf{V}_{tr}\}|$ nodes. Thus, the OCGNN optimization problem for r can be written as

$$\begin{aligned} & \underset{r}{\operatorname{argmin}} r^2 + \frac{1}{\beta K} \sum_{v_i \in \mathbf{V}_o} (d_{v_i} - r^2) + \xi \\ &= \underset{r}{\operatorname{argmin}} \left(1 - \frac{K_o}{\beta K}\right) r^2 + \frac{1}{\beta K} \sum_{v_i \in \mathbf{V}_o} d_{v_i} + \xi \\ &\Rightarrow \underset{r}{\operatorname{argmin}} \left(1 - \frac{K_o}{\beta K}\right) r^2. \end{aligned} \quad (4)$$

Since the second and third term of Eq. (4) is nonnegative, radius r is decreased as long as $K_o \leq \beta K$. That is, $\frac{K_o}{K} \leq \beta$ must satisfy in the optimum, which means β is an upper bound on the fraction of anomalous training nodes. The optimal radius r^* is determined by the largest K_o that satisfies the inequality, because if the inequality is not satisfied, the model cannot continue to minimize r . Hence, we also have $\frac{K_o+1}{K} > \beta$. β is a lower bound on the fraction of samples being outside of the hypersphere. β allows some nodes to be mapped out of the sphere, otherwise, in order to enclose all training nodes, the radius r will be so large that OCGNN will fail to detect outliers.

3.4 The paradigms of OCGNN

Our OCGNN is a GNN-based GAD framework, therefore OCGNN can be formed by any suitable GNN layer, such as Graph Convolutional network (GCN) [14], Graph Attention Network (GAT) [33] and GraphSAGE [11]. In this section, we use GCN as an example to illustrate how OCGNN learns the node representation. If other forms of GNN layers are considered, people just need to replace Eq. (5) by other neighbor nodes aggregation function. GCN is one of the best paradigms of graph learning model. It is a multilayer GNN model which expands Eq. (2) as follows:

$$g(\mathbf{H}^{(l)}, \mathbf{A}; \mathbf{W}^{(l)}) = \sigma(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)}), \quad (5)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$ is the adjacency matrix of a graph whose nodes are added self-loop connections. $\tilde{\mathbf{D}}$ denotes the diagonal matrix of $\tilde{\mathbf{A}}$, where $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$. $\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$ is the symmetric normalized adjacency matrix $\hat{\mathbf{A}}$, so we can directly compute $\hat{\mathbf{A}}$ as a pre-processing step. $\sigma(\cdot)$ is a nonlinear activation function, such as ReLU. Note that the weight matrix $\mathbf{W}^{(l)}$ is shared for all nodes on the given graph. From the view of spectral graph theory [6], GCN learns the node embedding $\mathbf{Z}_i$ by aggregating its own features $\mathbf{X}_i$ and first-order neighbor's features $\mathbf{X}_j$, where $j \in N(v_i)$, but the information of k th-order neighborhood can be captured by stacking multiple GCN layers.

3.5 Optimization

The optimization process of OCGNN is summarized in Algorithm 1. A trained OCGNN model can be characterized by three parameters: weight matrix $\mathcal{W}$, radius r and data center c . Like other neural network models, we use stochastic gradient descent to optimize the parameters $\mathcal{W}$ of the GNN model with OCGNN objective by back propagation (BP). Since radius r is not an inner parameter of OCGNN network, r and $\mathcal{W}$ cannot be optimized synchronously by BP algorithm. Instead, we update the r and $\mathcal{W}$ alternately during the training phase. First, we train $\mathcal{W}$ for $\phi \in \mathcal{N}$ epochs while r is fixed. After every ϕ th epochs, r can be solved by simple linear percentile search. That is, for the training node set $\mathbf{V}_{tr}$, we can obtain a distance set $\mathbf{d}_{\mathbf{V}_{tr}} \in \mathcal{R}^K$. Afterward, we can sort $\mathbf{d}_{\mathbf{V}_{tr}}$ from small to large, and the radius r can be defined via $(1 - \beta)$ percentile of $\mathbf{d}_{\mathbf{V}_{tr}}$. Empirically, OCGNN is not sensitive to this parameter ϕ . During the training phase, $c \in \mathcal{R}^F$ is fixed as c_0 , which is computed from the mean of the training nodes embeddings by an initial forward propagation. To map the nodes around a target data center, c_0 is a good choice because most nodes are not too far away from c_0 in the node embedding space.

Algorithm 1 Training OCGNN model

Input: Attributed Graph $\mathcal{G} = (\mathbf{V}, \mathbf{E}, \mathbf{X})$, normal nodes set $\mathbf{V}_{tr}$, Slack parameter $\beta \in (0, 1]$, weight decay $\lambda > 0$

Output: Weights $\mathcal{W}$, center $c \in \mathcal{R}^F$ and radius $r \in \mathcal{R}^+$

```

1: Initialize  $\mathcal{W}$  using Glorot uniform initialization.
2: Initialize  $r = 0, c = \frac{1}{K} \sum_{v_i \in \mathbf{V}_{tr}} g(\mathbf{X}_i, \mathbf{A}_i; \mathcal{W})_{v_i}$ .
3: while epoch < max epoch budget do
4:    $\mathbf{d}_{\mathbf{V}_{tr}} = \left\| g(\mathbf{X}, \mathbf{A}; \mathcal{W})_{\mathbf{V}_{tr}} - c \right\|^2, \mathbf{d}_{\mathbf{V}_{tr}} \in \mathcal{R}^K$ 
5:    $\mathcal{L} = \frac{1}{\beta K} \sum_{\mathbf{V}_{tr}} [\mathbf{d}_{\mathbf{V}_{tr}} - r^2]^+ + r^2 + \frac{\lambda}{2} \sum_{l=1}^L \left\| \mathbf{W}^{(l)} \right\|^2$ 
6:   Update  $\mathcal{W}$  by its stochastic gradient  $\nabla_{\mathcal{W}}(\mathcal{L})$ 
7:   if epoch mod  $\phi = 0$  then
8:     Update  $r$  using  $(1 - \beta) \times 100\%$  percentile of  $\mathbf{d}_{\mathbf{V}_{tr}}$ 
9:   end if
10: end while
11: return  $\mathcal{W}, c$  and  $r$ 

```

4 Experiments

In this section, we introduce the detailed experimental setup and results, including datasets, baselines, network structures, hyperparameter selections and performance analysis. The source code of our models is implemented using PyTorch and Deep Graph Library [37], which has been released at a Github repository² for reproducibility. Besides, in our repository, we implement six network embedding modules, four anomaly detection modules and four graph neural network modules, as well as a function for generating GAD datasets from common node classification dataset. All these functions can be run automatically with simple python commands.

4.1 Datasets

Cora, Citeseer and Pubmed [31] are publicly available and broadly used citation network datasets in previous studies [11, 14, 33]. Scientific publications and their citation relationship are represented as nodes and edges on a graph, respectively. The node feature of each publication is described by a sparse bag-of-words feature vector computed from a dictionary. Our node anomaly detection datasets used in this paper are generated from these three plain node classification datasets by regarding one class as normal and the rest as anomalous. The normal classes in Cora, Citeseer and Pubmed datasets are “Neural Networks,” “IR” and “Diabetes Mellitus Type 2,” respectively. All the nodes in the train set pertain to the normal class, while, in the validation set and the test set, half of the nodes are normal and the other half are randomly sampled from the anomalous class. The characteristics of all three datasets are summarized in Table 2, and our Github repository provides more details about these three GAD

datasets. For all the methods, we use in this paper, the hyperparameters are tuned using the validation set, then the performances are evaluated in the test set.

4.2 Baselines

To the best of our knowledge, our model is the first end-to-end approach that applies the hypersphere learning objective to anomaly detection task in attributed network. Since hypersphere learning is one of the most important anomaly detection methods, its application to graph data is an inevitable evolution. Therefore, two-stage methods are widely used to concatenate the graph embedding methods and anomaly detection methods. In the first stage, graph embedding learns and maps the graph structure information into a fixed-length embedding vector. For the second stage, Euclidean anomaly detection methods can be trained by the embedding vector and node feature vector.

We choose three popular graph embedding methods (DeepWalk [26], Struc2Vec [28] and SDNE [35]) to learn a 128-dimensional embedding vector as the first stage method. DeepWalk randomly samples node walk series and adopts a neural language model to learn a graph embedding vector. Struc2Vec measures local graph similarity at different scales and constructs a multilayer graph to generate structural context for nodes, and Struc2Vec is considered to be one of the best embedding methods for preserving graph structure. SDNE learns node

Table 2 Summary of the datasets used in our experiments

Datasets	Nodes	Edges	Features	Train/Val/Test nodes
Cora	2708	5429	1433	490/246/410
Citeseer	3327	4732	3703	420/210/352
Pubmed	19,717	44,338	500	4725/2364/3936

² <https://github.com/WangXuhongCN/OCGNN>.

representations that preserve the proximity between 1-hop and 2-hop neighbors with a deep AE. We build a three-layers SDNE with hidden size of 256-128-256. The “walk length,” “window size” and “walks per vertex” of DeepWalk are set as 5, 10 and 50, respectively, while for Struc2Vec, these three parameters are 10, 5, 80.

Four popular anomaly detection methods are used as the second stage and tuned by the performance in validation set: (i) OCSVM is our source method which we have described above. It has an important parameter β , which is chosen to be the same as in our OCGNN model. (ii) Isolation Forest (IForest) [17] is an efficient method of detecting anomaly data in high-dimensional datasets. IF recursively and randomly splits the data feature and stores this in a forest data structure. Outliers are defined as the data who have shorter path in the tree because it means the data need a smaller number of splittings from other majority data. (iii) PCA [21] and AE, two popular reconstruction-based approaches, assume that anomalies are incompressible and thus cannot be effectively reconstructed from low-dimensional projections.

Two SOTA GAE-based GAD methods are Dominant (DOM) [7] and a plain GAE [13]. DOM is a recently proposed SOTA unsupervised GAD method on attribute graph. We reproduced DOM using the hyperparameter recommended by the authors and added the early stop technology. All these models choose GCN layers to construct the encoder part of GAE, and the main difference between GAE and DOM is the decoder structure as well as the training objective. Plain GAE proposed an inner product decoder to predict the unseen links on the graph, which equals to reconstruct the connection relationship between nodes. DOM designs an additional GCN-based decoder aiming at reconstructing the node attributes, so DOM has two different decoders for reconstructing features and links, respectively, and a trade-off controlling parameter (set to be 0.5 as authors recommended) between them. DOM, GAE and our OCGNN share the same encoder structure, while DOM also has an inverse decoder and an inner product decoder.

4.3 OCGNN setup

Our OCGNN is a GNN framework for GAD, and it can be formed by any suitable GNN layer. To evaluation the performance of OCGNN, we use three popular GNN layer paradigms, GCN [14], GAT [33] and GraphSAGE [11], whose model configurations are almost same. To better demonstrate the true performance of the OCGNN framework, we use the same hyperparameters for training in all datasets and OCGNN paradigms. The penalty parameter β is set as 0.1. The OCGNN model is

initialized with Glorot uniform weight initialization [9] and optimized by the AdamW [19] SGD optimizer with a learning rate of 0.001. During training, we apply weight decay regularization with $\lambda = 0.0005$. We trained each OCGNN model using an early stopping strategy on both the OCGNN loss [Eq. (3)] and AUC score on the validation set, with a maximum of 5000 epochs and a patience of 100 epochs. For Cora and Citeseer datasets, we apply a three-layers OCGNN with the hidden size of 64-64-32. For Pubmed dataset, we use a two-layers OCGNN with hidden size of 128-64. For each individual OCGNN model, the dimension of output layer (the dimension of node embedding) is always the half of the hidden size. After each GNN layer, the dropout layer of 0.5 rate and the ReLU activation function is applied. The aggregator type of each GraphSAGE layer is set as **pooling**, and the attention heads of GAT layer is set as 8. Note that the structure and hyperparameters used in this study proved to be sufficient for our applications, although they can still be improved.

4.4 Results

Table 3 shows the experiment results at AUC metrics. According to the experiment results, our OCGNN model achieves the best results in all datasets. Among three variants of OCGNNs, we believe that OC-SAGE is the most powerful and stable one especially in larger datasets like Pubmed, because OC-SAGE has great robustness because of its simple neighbor aggregation strategy. OC-GAT performs well in the small Cora dataset but lost superiority in Pubmed. These two models have the same hidden dimension, but OC-GAT has more complicated attention mechanism, which needs more hidden neurons to capture the common patterns of a larger dataset. There is no significant performance gap between OC-GCN and DOM, but OC-GCN has fewer net parameters and lower time complexity than DOM, which we will explain in next section.

Comparing among four anomaly detection methods, only AE and PCA achieve competitive results with raw features. The reason is that PCA and AE are better at processing high-dimensional original features rather than reconstructing sparse adjacency vectors. IForest and OCSVM fail because they are not good at handling high-dimensional data. While methods such as DeepWalk can provide low-dimensional embedding, that embedding suffers from problems that are irrelevant to the GAD task. Comparing among three graph embedding methods, we can conclude that different network embedding methods have little effect on performance, which further demonstrates

Table 3 For each dataset and method, we show its average AUCs in % with StdDevs (over 10 random seeds) and average rank with StdDevs

Available Data	Methods		Cora	Citeseer	Pubmed	Avg. Rank
X	IForest		53.09 ± 0.03	46.33 ± 0.03	65.57 ± 0.02	23.7 ± 10.1
	OCSVM		54.35 ± 0.02	57.05 ± 0.03	45.50 ± 0.01	24.0 ± 9.8
	PCA		62.17 ± 0.01	58.10 ± 0.03	71.06 ± 0.01	7.0 ± 3.0
	AE		62.17 ± 0.01	58.11 ± 0.03	71.05 ± 0.01	7.0 ± 2.6
A	DeepWalk	IForest	57.87 ± 0.02	51.00 ± 0.03	60.73 ± 0.01	17.3 ± 3.5
		OCSVM	52.10 ± 0.03	43.13 ± 0.02	60.22 ± 0.01	27.0 ± 6.9
		PCA	55.90 ± 0.03	46.65 ± 0.02	61.66 ± 0.01	19.3 ± 6.0
		AE	55.91 ± 0.03	46.42 ± 0.02	61.66 ± 0.01	19.3 ± 7.1
	Struc2Vec	IForest	56.07 ± 0.02	54.07 ± 0.04	48.53 ± 0.00	20.3 ± 6.7
		OCSVM	55.95 ± 0.02	55.45 ± 0.02	48.27 ± 0.00	21.0 ± 8.7
		PCA	55.82 ± 0.02	53.60 ± 0.03	48.57 ± 0.00	22.7 ± 4.5
		AE	55.80 ± 0.02	53.02 ± 0.03	48.38 ± 0.01	24.0 ± 5.0
	SDNE	IForest	57.87 ± 0.02	51.00 ± 0.03	60.73 ± 0.01	17.3 ± 3.5
		OCSVM	52.10 ± 0.03	43.13 ± 0.02	60.22 ± 0.01	27.0 ± 6.9
		PCA	55.90 ± 0.03	46.65 ± 0.02	61.66 ± 0.01	19.3 ± 6.0
		AE	55.91 ± 0.03	46.42 ± 0.02	61.66 ± 0.01	19.3 ± 7.1
	DeepWalk + Features	IForest	53.56 ± 0.04	45.55 ± 0.06	65.60 ± 0.02	23.3 ± 10.7
		OCSVM	51.59 ± 0.03	42.95 ± 0.02	60.10 ± 0.01	29.0 ± 6.9
		PCA	62.38 ± 0.02	57.96 ± 0.03	72.04 ± 0.01	6.3 ± 3.8
		AE	62.39 ± 0.02	57.96 ± 0.03	71.91 ± 0.01	6.3 ± 3.1
	Struc2Vec + Features	IForest	53.92 ± 0.03	50.72 ± 0.05	58.48 ± 0.01	24.3 ± 3.2
		OCSVM	55.07 ± 0.02	56.03 ± 0.02	48.35 ± 0.01	23.0 ± 8.2
		PCA	63.08 ± 0.01	57.90 ± 0.01	68.00 ± 0.01	7.7 ± 3.1
		AE	62.83 ± 0.01	57.70 ± 0.01	68.02 ± 0.01	8.0 ± 3.5
	SDNE + Features	IForest	55.07 ± 0.02	50.47 ± 0.05	54.90 ± 0.02	24.3 ± 1.0
		OCSVM	55.85 ± 0.03	54.97 ± 0.02	45.47 ± 0.01	23.7 ± 8.6
		PCA	61.85 ± 0.02	58.03 ± 0.01	66.10 ± 0.01	9.7 ± 2.1
		AE	62.30 ± 0.02	58.37 ± 0.01	65.97 ± 0.01	8.0 ± 2.6
A + X	GAE [13]		60.15 ± 0.18	51.80 ± 0.03	54.27 ± 0.02	19.3 ± 6.0
	DOM [7]		64.50 ± 0.25	62.44 ± 0.15	50.92 ± 0.04	11.3 ± 12.7
	OC-GCN		73.25 ± 0.02	62.81 ± 0.01	54.53 ± 0.01	10.0 ± 12.1
	OC-GAT		88.19 ± 0.02	79.06 ± 0.03	66.98 ± 0.01	3.7 ± 3.8
	OC-SAGE		86.97 ± 0.04	85.62 ± 0.01	74.72 ± 0.03	1.3 ± 0.6

Note that the best result is typeset in bold. OC-GAT and OC-SAGE achieved the best rankings and performed consistently across the three datasets

that the two-stage approach fails empirically in the GAD task.

OCGNN outperforms the other two-stage methods, reaching the highest AUC improvement of above 30% than OCSVM does in Cora and Citeseer dataset. Besides, we can conclude that DOM (reconstructing node attributes and adjacency vectors) and GAE (reconstructing the adjacency vectors only) have no significant performance difference, because the loss value of the two objectives in DOM may not be on the same order of magnitude, which can cause instabilities during training.

We performed a T-SNE [20] visualization analyses on the embeddings learned by the OCGNN framework in order to better understand its properties. We focus our analysis exclusively on the Cora dataset and use the GraphSAGE algorithm to form our OCGNN. The reasons are that Cora dataset has the smallest number of nodes which can significantly aid clarity and GraphSAGE is an outstanding graph learning layer that is simple in form and has fewer hyperparameters. The embedding visualization in Fig. 2a, b indicates that we cannot detect anomalous nodes by the raw node features or the initialized OCGNN. After 500 epochs of training, the normal and anomalous

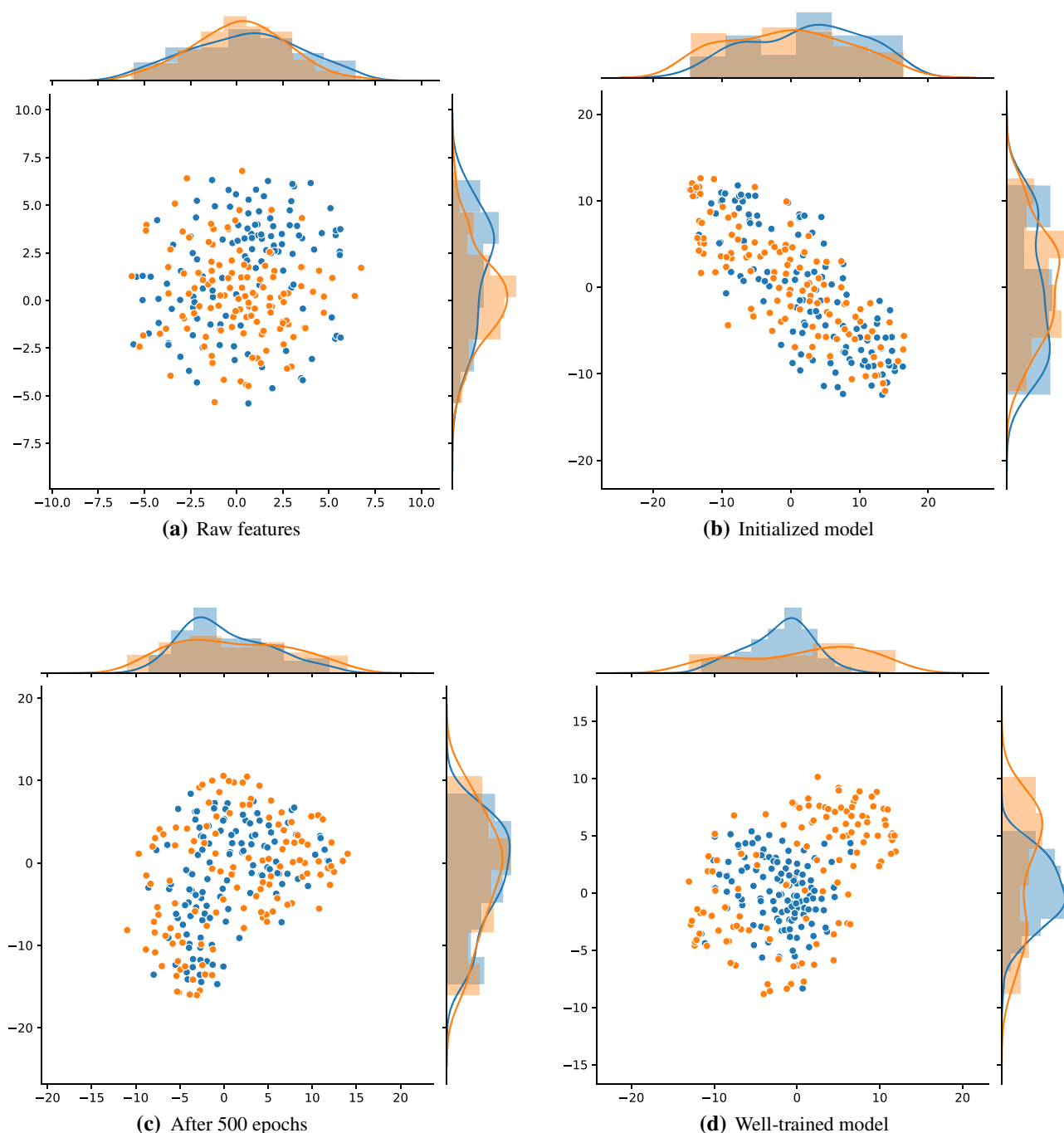


Fig. 2 T-SNE visualization of the node embeddings in Cora dataset. The KDE curves at top and right show the probability distribution in each dimension. Blue and orange points represent the normal nodes and the anomalous nodes, respectively. **a** The visualization of nodes' raw features. **b** The visualization of node embeddings from a

randomly initialized OCGNN model. **c** Node embeddings after 500 epochs training. **d** Node embeddings from a learned OCGNN model. We can conclude that OCGNN does learn the features of normal nodes and can distinguish abnormal nodes in an unsupervised manner

nodes seem to have a tendency to separate. The KDE curves in Fig. 2d prove that where the probability density of normal nodes is high, the density of anomalous nodes is low, and vice versa. It means that a majority of normal and anomalous nodes can be easily partitioned in the embedding space, if the model converges.

4.5 Robustness

To verify the impact of network structure on performance, we tune the number of hidden layers from 1 to 7 and adjust the number of hidden neurons in each hidden layer from 16 to 128, respectively. The results from different network

Fig. 3 The heatmap of average AUCs (%) of diverse OCGNN models in Cora dataset. The horizontal axis represents the dimensions of the hidden layer, and the vertical axis represents the number of layers in the network. The lighter the color, the higher the AUC performance of the model. Basically, the performance of the OCGNN model is robust to the structure of the network

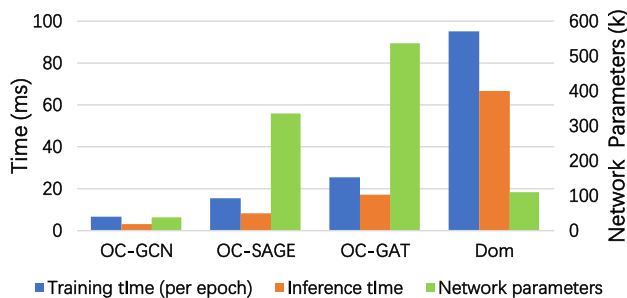
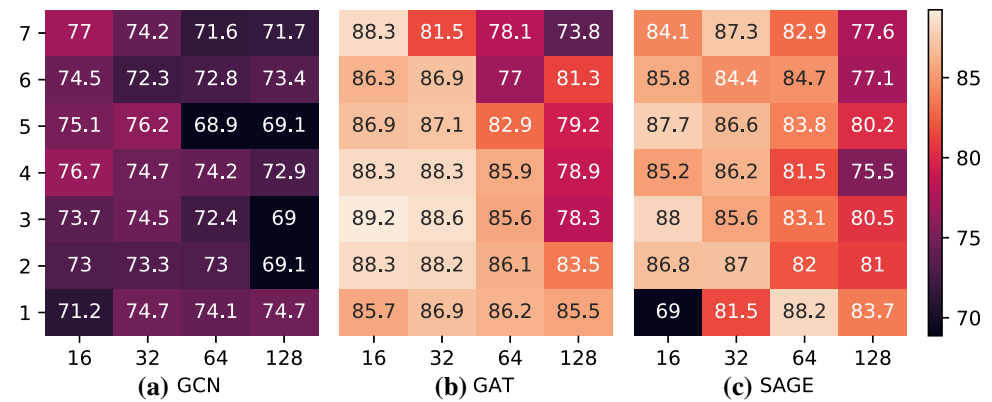


Fig. 4 Runtime experiments on the Pubmed dataset. It shows the training and test runtimes and number of network parameters for the different approaches. DOM has three times as many net parameters than that of OC-GCN, and 15 times slower than OC-GCN, whereas OC-SAGE and OC-GAT have above ten times as many parameters but do not induce much increase in runtime

structures are shown in Figure 3 as heatmaps. OCGNN's performance will decrease when the size of the network is too large (top right) or too small (bottom left). The best results are usually obtained with a 2- to 4-layer model and 16 or 32 dimensions. When the network capacity is extremely small (1 hidden layer and 16 dimensions), the performance of OCGNN is greatly reduced because the insufficient network capacity cannot learn enough information from the normal training data. On the other hand, OCGNNs achieve slightly worse results when the network is too deep or too wide. A deep GNN model usually suffers from the over-smooth problem [39] because GNN models iteratively aggregate the node information of almost the entire graph layer by layer, and calculate node embeddings of low diversity. If the network is too wide, curse of dimensionality will cause OCGNN to fail in calculating distance terms in the loss function, because the distances of data points are approaching the same in high-dimensional space. As long as the scale of neural network is large enough, the number of network layers or the number of neurons will not have a significant impact on the results.

4.6 Efficiency

We conducted the runtime experiments on a Linux server with an Intel Xeon CPU and an NVIDIA GeForce GTX 1080Ti GPU. Figure 4 summarizes the training and test runtimes and number of network parameters for the different approaches. GCN-based models have fewer network parameters but still result in worse AUC performance, as shown in Table 3. DOM has three times as many net parameters than OC-GCN because DOM is a three-network model: an encoder, an attribute decoder and a structure decoder, while our OCGNN only has one encoder. This means that DOM has three times as many layers as the OCGNN model does, which will increase the gradient BP time. Besides that, the cost function of DOM is to compute the mean square error between high-dimensional input (X and A) and their reconstruction matrix, whereas OCGNN's loss is a computation between a low-dimensional embedding matrix Z and a data center vector c . Therefore, the time complexity of DOM is significant higher than our OCGNN model. Although the network scales of OC-SAGE and OC-GAT are much more larger than OC-GCN, they only increase the time complexity by a few times, because these three OCGNN models have the same numbers of GNN layer and DOM have almost three times numbers of layer, which is the core factor of time complexity.

In order to provide readers with richer information of time efficiency, we take DeepWalk + OCSVM as an example to report the efficiency of traditional anomaly detection methods. In Pubmed dataset, the time cost of DeepWalk pretraining is 218,877 ms, and the training and inference time of OCSVM are 18,884 and 3,309 ms, respectively. GNNs are 7000× faster than DeepWalk, because graph embedding methods often adopt heavier random walk operation, while GNNs use lighter neighbors sampling. Besides, OCGNNs are 600× faster than OCSVM, because neural network methods can use GPU to process large-scale data in parallel manner, while

traditional machine learning models can only process data serially.

5 Conclusion

In this paper, we proposed one-class graph neural network (OCGNN), a one-class classification framework for graph anomaly detection. OCGNN aims at mapping the training nodes into a hypersphere in the embedding space, through the powerful representation ability of graph neural networks. Our results from extensive experiments demonstrate that the proposed OCGNN achieves significant improvements. In the future, we will generalize OCGNN into graph-level and dynamic node anomaly detection.

Acknowledgements This research is supported by National Natural Science Foundation of China (Nos. 51777122 and 61273161). This work is also supported by the National Research Foundation of Singapore through a grant to the Berkeley Education Alliance for Research in Singapore (BEARS) for the Singapore-Berkeley Building Efficiency and Sustainability in the Tropics (SinBerBEST) program, and by the Defence Science & Technology Agency (DSTA) of Singapore.

Declarations

Conflicts of interest The authors declare that they have no conflict of interest.

References

- Akoglu L, Tong H, Koutra D (2015) Graph based anomaly detection and description: a survey. *Data Min Knowl Discov* 29(3):626–688
- Bresson X, Laurent T (2017) Residual gated graph convnets. *CoRR arXiv:1711.07553*
- Breunig MM, Kriegel H, Ng RT, Sander J (2000) LOF: identifying density-based local outliers. In: *ACM SIGMOD international conference on management of data (SIGMOD)*, ACM, pp 93–104
- Candès EJ, Li X, Ma Y, Wright J (2011) Robust principal component analysis? *J ACM* 58(3):11:1–11:37
- Chen J, Sathe S, Aggarwal CC, Turaga DS (2017) Outlier detection with autoencoder ensembles. In: *SDM, SIAM*, pp 90–98
- Defferrard M, Bresson X, Vandergheynst P (2016) Convolutional neural networks on graphs with fast localized spectral filtering. In: *NeurIPS*, pp 3837–3845
- Ding K, Li J, Bhanushali R, Liu H (2019) Deep anomaly detection on attributed networks. In: *SDM, SIAM*, pp 594–602
- Gao J, Liang F, Fan W, Wang C, Sun Y, Han J (2010) On community outliers and their efficient detection in information networks. In: *KDD, ACM*, pp 813–822
- Glorot X, Bengio Y (2010) Understanding the difficulty of training deep feedforward neural networks. In: *AISTATS, JMLR.org*, vol 9, pp 249–256
- Goodfellow IJ, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville AC, Bengio Y (2014) Generative adversarial nets. In: *Annual conference on neural information processing systems (NeurIPS)*. MIT Press, pp 2672–2680
- Hamilton WL, Ying Z, Leskovec J (2017) Inductive representation learning on large graphs. In: *NIPS*, pp 1024–1034
- Kingma DP, Welling M (2014) Auto-encoding variational bayes. In: *International conference on learning representations (ICLR)*
- Kipf TN, Welling M (2016) Variational graph auto-encoders. *CoRR arXiv:1611.07308*
- Kipf TN, Welling M (2017) Semi-supervised classification with graph convolutional networks. In: *ICLR (Poster)*, OpenReview.net
- Li J, Dani H, Hu X, Liu H (2017) Radar: residual analysis for anomaly detection in attributed networks. In: *IJCAI, ijcai.org*, pp 2152–2158
- Li Y, Huang X, Li J, Du M, Zou N (2019) Specac: spectral autoencoder for anomaly detection in attributed networks. In: *CIKM, ACM*, pp 2233–2236
- Liu FT, Ting KM, Zhou Z (2008) Isolation forest. In: *ICDM*. IEEE Computer Society, pp 413–422
- Liu Y, Li Z, Zhou C, Jiang Y, Sun J, Wang M, He X (2020) Generative adversarial active learning for unsupervised outlier detection. *IEEE Trans Knowl Data Eng* 32(8):1517–1528
- Loshchilov I, Hutter F (2019) Decoupled weight decay regularization. In: *ICLR*
- Maaten L, Hinton G (2008) Visualizing data using t-SNE. *J Mach Learn Res* 9:2579–2605
- Olive DJ (2017) *Principal component analysis*. Springer, New York, pp 189–217
- Peng H, Li J, Gong Q, Song Y, Ning Y, Lai K, Yu PS (2019) Fine-grained event categorization with heterogeneous graph convolutional networks. In: *IJCAI, ijcai.org*, pp 3238–3245
- Peng Z, Luo M, Li J, Liu H, Zheng Q (2018) ANOMALOUS: a joint modeling approach for anomaly detection on attributed networks. In: *IJCAI, ijcai.org*, pp 3513–3519
- Perozzi B, Akoglu L (2016) Scalable anomaly ranking of attributed neighborhoods. In: *SDM, SIAM*, pp 207–215
- Perozzi B, Akoglu L, Sánchez PI, Müller E (2014a) Focused clustering and outlier detection in large attributed graphs. In: *KDD, ACM*, pp 1346–1355
- Perozzi B, Al-Rfou R, Skiena S (2014b) Deepwalk: online learning of social representations. In: *KDD, ACM*, pp 701–710
- Pimentel MAF, Clifton DA, Clifton LA, Tarassenko L (2014) A review of novelty detection. *Signal Process* 99:215–249
- Ribeiro LFR, Saverese PHP, Figueiredo DR (2017) *struc2vec*: learning node representations from structural identity. In: *KDD, ACM*, pp 385–394
- Ruff L, Görnitz N, Deecke L, Siddiqui SA, Vandermeulen RA, Binder A, Müller E, Kloft M (2018) Deep one-class classification. *ICML, PMLR, Proc Mach Learn Res* 80:4390–4399
- Sánchez PI, Müller E, Laforet F, Keller F, Böhm K (2013) Statistical selection of congruent subspaces for mining attributed graphs. In: *ICDM, IEEE Computer Society*, pp 647–656
- Sen P, Namata G, Bilgic M, Getoor L, Gallagher B, Eliassi-Rad T (2008) Collective classification in network data. *AI Maga* 29(3):93–106
- Tax DMJ, Duin RPW (2004) Support vector data description. *Mach Learn* 54(1):45–66
- Velickovic P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y (2018) Graph attention networks. In: *ICLR (Poster)*, OpenReview.net
- Wang C, Wang J, Wang C, Shen Q (2018) Actor model anomaly detection using kernel principal component analysis. In: *ICONIP (4)*, Lecture Notes in Computer Science, vol 11304. Springer, pp 545–554
- Wang D, Cui P, Zhu W (2016) Structural deep network embedding. In: *KDD, ACM*, pp 1225–1234

36. Wang D, Qi Y, Lin J, Cui P, Jia Q, Wang Z, Fang Y, Yu Q, Zhou J, Yang S (2019a) A semi-supervised graph attentive network for financial fraud detection. In: ICDM, IEEE, pp 598–607
37. Wang M, Yu L, Zheng D, Gan Q, Gai Y, Ye Z, Li M, Zhou J, Huang Q, Ma C, Huang Z, Guo Q, Zhang H, Lin H, Zhao J, Li J, Smola AJ, Zhang Z (2019b) Deep graph library: towards efficient and scalable deep learning on graphs. In: ICLR workshop on representation learning on graphs and manifolds
38. Wang X, Du Y, Lin S, Cui P, Shen Y, Yang Y (2020) advae: a self-adversarial variational autoencoder with gaussian anomaly prior knowledge for anomaly detection. *Knowl Based Syst* 190:105187
39. Xu K, Li C, Tian Y, Sonobe T, Kawarabayashi K, Jegelka S (2018) Representation learning on graphs with jumping knowledge networks. *ICML, PMLR, Proc Mach Learn Res* 80:5449–5458
40. Xu K, Hu W, Leskovec J, Jegelka S (2019) How powerful are graph neural networks? In: ICLR, OpenReview.net
41. Yu B, Yin H, Zhu Z (2018) Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In: IJCAI, ijcai.org, pp 3634–3640
42. Zhang H, Wang S, Xu X, Chow TWS, Wu QMJ (2018) Tree2vector: learning a vectorial representation for tree-structured data. *IEEE Trans Neural Netw Learn Syst* 29(11):5304–5318
43. Zhou C, Paffenroth RC (2017) Anomaly detection with robust deep autoencoders. In: KDD, ACM, pp 665–674

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.